Temporal Event Evaluation: Soft F1, Tolerance, and Detection Delay

CEFET/RJ

Data Analytics Lab

dal.global@grupo.cefet-rj.br | https://eic.cefet-rj.br/dal

Evaluating temporal events is not classifying points

The central axis of this presentation: evaluating event detection is not the same as classifying independent instants.

1 **The temporal position of the detection matters**

A nearby detection can be useful even without an exact match.

2 **Events have distinct natures**

They can be point-based, interval-based, anticipatable, or only detectable with a delay.

3 **Classical metrics ignore temporal structure**

We need specific approaches for time series.

The problem of exact matching

Classical metrics treat each instant as independent — and this has serious consequences for evaluating detectors.

Independent instants

Classical metrics treat each instant as independent, without considering temporal neighborhood.

Double penalty

An error of one time unit simultaneously generates a false positive and a false negative.

Useful detectors discarded

This can eliminate operationally valuable detectors in real-world applications.

Events as the Positive Class

Instant-level Labeling

Each instant in the series is labeled as an **event (positive)** or **non-event (negative)**.

Events: Positive Class

Events are the positive class — **rare**, but of **high interest**.

Typical Observations: Negative Class

Typical observations are the negative class — **dominant in volume**, but of lesser direct interest.

This class imbalance is central to understanding why **accuracy** can be misleading.

Confusion Matrix

TP — True Positive

Real event correctly detected.

TN — True Negative

Non-event correctly ignored.

FP — False Positive

Alarm triggered where no real event occurred.

FN — False Negative

Real event not detected by the method.

TP and TN are correct predictions. FP and FN are errors with distinct consequences.

Accuracy and Error Rate

Accuracy

Total proportion of correct predictions over all observations.

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Error Rate

Complement of accuracy.

$$\text{error rate} = 1 - \text{accuracy} = \frac{FP + FN}{TP + TN + FP + FN}$$

Limitation with rare events

- In series with few events, correctly predicting all non-events already ensures high accuracy.
- The method can have zero recall and still present high accuracy.
- Accuracy alone is not sufficient to evaluate event detectors.

Precision: accuracy of detections

Precision measures, among everything the detector flagged as an event, what fraction was an actual event.

$$\text{precision} = \frac{TP}{TP + FP}$$

Parameters

- **TP** = correct detections
- **FP** = false alarms

When it matters

- False alarms have a high cost (e.g., unnecessary production shutdown).
- Low precision indicates an excess of false positives.

Interpretation

Precision = 1 means every detection was a real event.
Precision = 0 means no detection was correct.

Recall: completeness of events

Recall measures, among all real events, what fraction was detected by the method.

$$\text{recall} = \frac{TP}{P} \quad \text{where } P = TP + FN$$

Parameters

- **TP** = detected events
- **FN** = missed events
- **P** = total real events

When it matters

- Missing an event is more serious than issuing an extra alarm (e.g., critical equipment failure).
- Low recall indicates that many events went undetected.

Interpretation

Recall = 1 means all events were detected. Recall = 0 means no events were found.

F1 and Fβ

F1 — Harmonic mean

Combines precision and recall in a balanced way.

$$F1 = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Fβ — Weighted version

Allows weighting recall more (β > 1) or less (β < 1) than precision.

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{precision} \cdot \text{recall}}{\beta^2 \cdot \text{precision} + \text{recall}}$$

Interpreting β

- **β = 1** → F1: equal weight for precision and recall.
- **β = 2** → recall has twice the weight of precision.
- **β = 0.5** → precision has twice the weight of recall.

When to use Fβ

When the cost of missing events (FN) and the cost of false alarms (FP) are asymmetric.

Specificity and Fallout

Specificity (True Negative Rate)

Measures the ability to correctly identify non-events.

$$\text{specificity} = \frac{TN}{TN + FP}$$

High specificity indicates few false positives.

Fallout (False Positive Rate)

Measures the proportion of non-events incorrectly flagged as events.

$$\text{fallout} = \frac{FP}{TN + FP} = 1 - \text{specificity}$$

High fallout indicates an excess of alarms during normal periods.

Specificity and fallout are complementary and fundamental for constructing the ROC curve.

Algorithm: Hard Point-by-Point Evaluation

Definition

Evaluates each time instant of the series independently, comparing the true label with the detector's decision.

Formulas

$$TP = \Sigma \mathbb{1}[real(t) = 1 \wedge det(t) = 1]$$

$$FP = \Sigma \mathbb{1}[real(t) = 0 \wedge det(t) = 1]$$

$$FN = \Sigma \mathbb{1}[real(t) = 1 \wedge det(t) = 0]$$

$$TN = \Sigma \mathbb{1}[real(t) = 0 \wedge det(t) = 0]$$

Parameters

- real(t) = true label at time instant t ∈ {0,1}
- det(t) = detector decision at time instant t ∈ {0,1}
- T = set of all time instants

Pseudocode

```
INPUT: series of true labels R[1..n], series of detections D[1..n]
PRE-CONDITION: |R| = |D| = n; R[t], D[t] ∈ {0, 1} for all t

STEP 1 — Initialize counters
  TP ← 0; FP ← 0; FN ← 0; TN ← 0
STEP 2 — Iterate over each time instant t from 1 to n
  IF R[t] = 1 AND D[t] = 1 → TP ← TP + 1
  IF R[t] = 0 AND D[t] = 1 → FP ← FP + 1
  IF R[t] = 1 AND D[t] = 0 → FN ← FN + 1
  IF R[t] = 0 AND D[t] = 0 → TN ← TN + 1
STEP 3 — Compute metrics
  precision ← TP / (TP + FP)   [if TP+FP > 0, else 0]
  recall    ← TP / (TP + FN)   [if TP+FN > 0, else 0]
  F1        ← 2·precision·recall / (precision+recall)

OUTPUT: TP, FP, FN, TN, precision, recall, F1
POST-CONDITION: TP + FP + FN + TN = n
```

Concrete Example (10 observations)

Series with 10 time instants, 5 true events (`R = [0,1,1,1,1,1,0,0,0,0]`). Detector emits 2 detections (`D = [0,0,1,0,0,0,0,1,0,0]`).

Result: TP=1 · TN=4 · FP=1 · FN=4. Accuracy=0.5 · precision=0.5 · recall=0.2 · F1=0.28.

Interpretation: Low recall reveals that 4 out of 5 events went undetected.

Event Score vs Binary Label

Binary Label

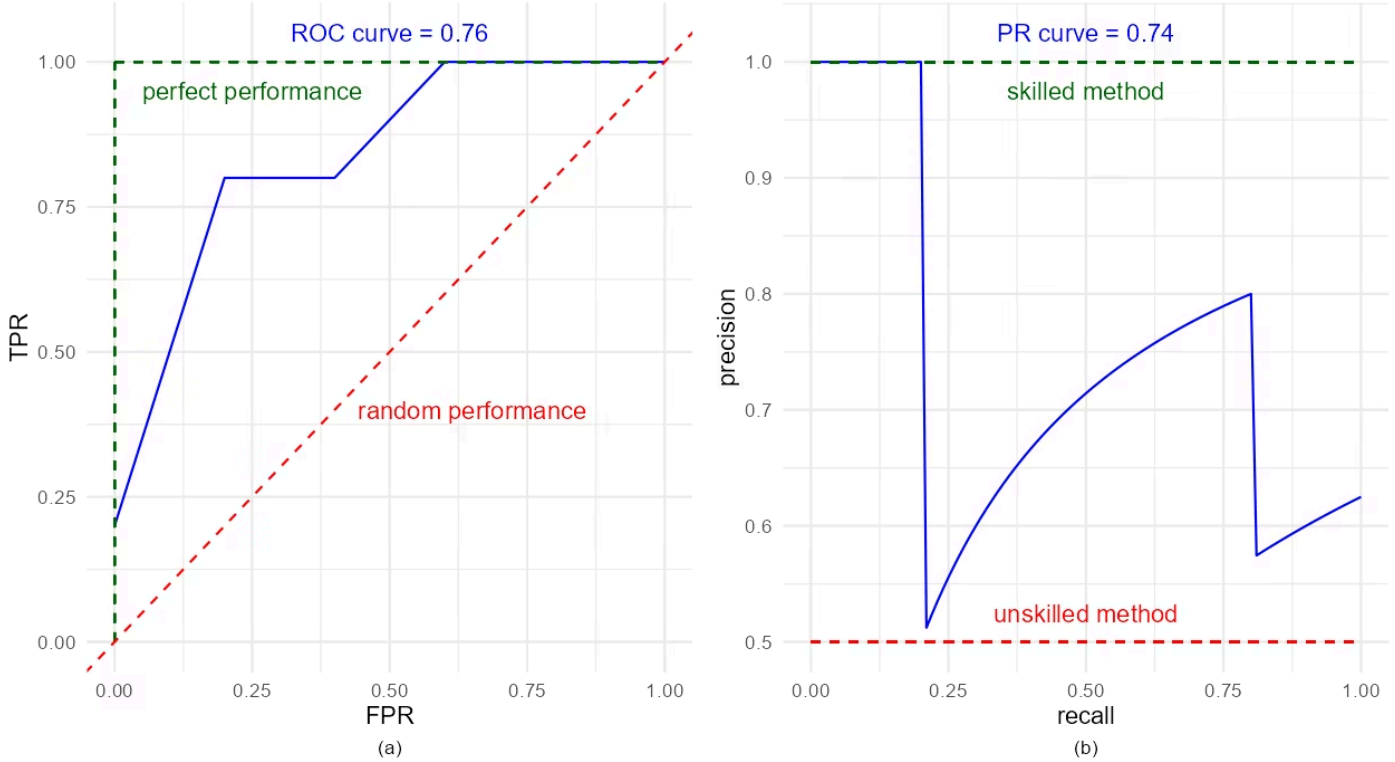
- The detector emits event or non-event for each time instant.
- Direct evaluation with confusion matrix.
- Does not allow varying the decision threshold.

Continuous Score

- The detector emits a degree of event evidence (score) for each time instant.
- The decision threshold can vary, generating different (precision, recall) or (fallout, recall) pairs.
- Allows building ROC and Precision-Recall curves.

The choice between label and score directly affects which evaluation metrics are applicable.

ROC, AUC and AUC-PR



ROC Curve

Plots recall (TPR) vs fallout (FPR) for all thresholds.

AUC-ROC: area under the ROC curve. Value 1 = perfect; 0.5 = random.

Precision-Recall Curve

Plots precision vs recall for all thresholds.

AUC-PR: more informative than ROC when events are rare.

Not affected by the large volume of true negatives.

Why classical metrics fail in time series

They ignore temporal order

Each time step is treated independently, without considering neighborhood.

They penalize neighboring detections

A detection close to the event is treated as a total error.

They miss essential dimensions

Delay, anticipation, persistence, and temporal coverage are ignored in real-world applications.

This motivates metrics with temporal tolerance and event-oriented evaluation.

The problem of the exact instant

Imprecise temporal boundary

Real events may not have a perfectly defined temporal boundary — the labeled instant is an approximation.

Effect vs cause

A detector may flag the effect of the event rather than the labeled instant, which can be operationally correct.

Excessive rigidity

Requiring an exact match between detection and label discards useful detectors and ignores the inherent uncertainty in labeling.

These problems motivate the use of temporal tolerance in evaluation.

Near miss is not a total error

A detection very close to the event can be operationally useful — but hard evaluation makes no such distinction.

Hard evaluation

Treats a near miss exactly like a distant error. Does not differentiate a detection 1 instant from the event from one 100 instants away. Result: operationally useful detectors are unfairly penalized.

What we need

A metric that rewards nearby detections in a controlled way, progressively penalizing detections that are farther away. This is the starting point for metrics with temporal tolerance.

Anticipated or Delayed Effect

Anticipatory Signals

Some events show precursor signals before the official timestamp — detecting early can be valuable.

Propagated Effects

Other events only become observable after their effects propagate — detection necessarily occurs with a delay.

Necessary Distinction

Evaluation must distinguish useful delay (inherent to the phenomenon) from irrelevant error (method failure).

Imprecision in Human Labels

1 **Label Sources**

Labels can be defined by experts, sensors, or external records — each source has its own degree of uncertainty.

2 **Annotation Imprecision**

Annotation may involve delay, rounding, or temporal uncertainty — the ground truth is not a perfect point in time.

3 **Tolerance as a Solution**

Temporal tolerance in evaluation accommodates the inherent imprecision of the labeling process.

Time Tolerance

Definition

Time tolerance k defines a time window around each event.

Detections within the window are considered close to the event.

Effect on Evaluation

Evaluation is no longer purely hard (hit or miss).

The temporal distance now influences the detection score.

Parameter k

- $k = 0$ → classic hard evaluation (no tolerance).
- $k > 0$ → tolerance window of k instants around the event.
- Choice of k depends on the domain and application.

Impact

With tolerance, nearby detections receive partial credit instead of a full penalty.

Algorithm: Applying Temporal Tolerance

Definition

Expands each real event into a tolerance window of k instants. Detections within the window are considered hits.

Window Formula

$$W(ej, k) = [ej - k, ej + k]$$

Hit Criterion with Tolerance

$$hit(tdi, ej, k) = \mathbb{I}[|tdi - ej| \leq k]$$

Parameters

- ej = instant of event j
- k = tolerance window (in instants)
- tdi = instant of detection i

Pseudocode

```
INPUT: events E[1..m], detections D[1..p], tolerance k ≥ 0
PRE-CONDITION: E and D are lists of instants; k ∈ ℕ

STEP 1 — For each event ej in E
  Create window W(ej) ← {t : |t − ej| ≤ k}
STEP 2 — For each detection tdi in D
  Check if tdi ∈ W(ej) for some ej in E
  IF yes → mark tdi as TP candidate for the closest ej
  IF no  → mark tdi as FP
STEP 3 — For each event ej in E
  IF some candidate detection exists for ej
    → ej is covered (contributes to TP)
  ELSE
    → ej not detected (contributes to FN)
STEP 4 — Compute metrics from the resulting counters

OUTPUT: list of associations (tdi → ej), TP, FP, FN
POST-CONDITION: each tdi associated with at most one ej
```

Detection Close to the Event

Controlled Reward

Temporal proximity should be rewarded proportionally — the closer the detection, the greater the credit.

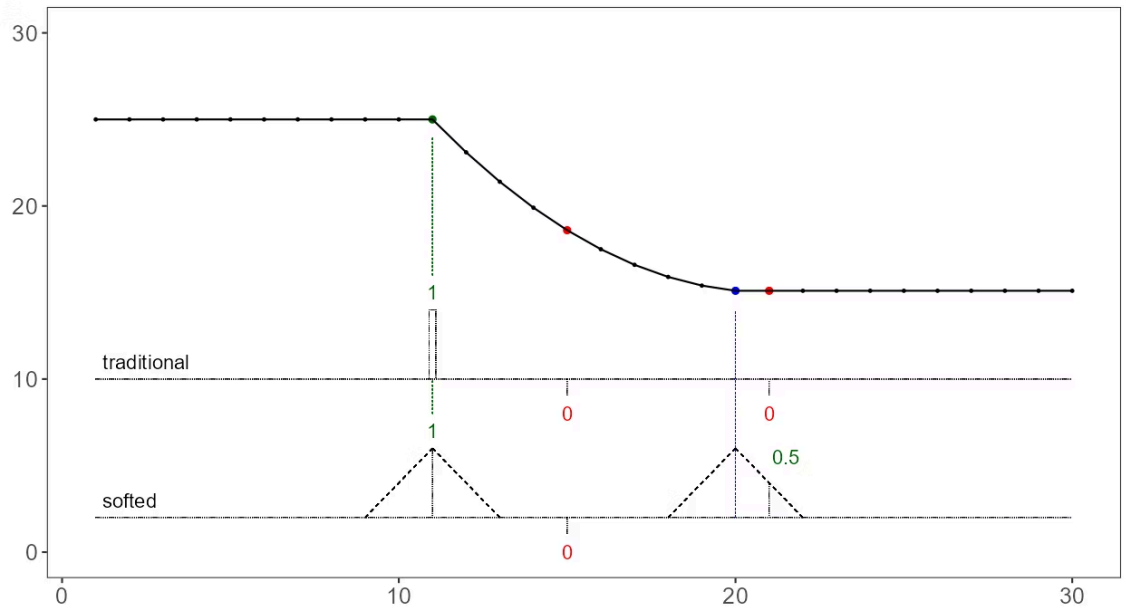
Progressive Penalty

Detections far from the event should receive no credit — the penalty grows with distance.

Tolerance Window

Outside the tolerance window k, the detection is treated as a total error, with no partial credit.

Hard evaluation vs Soft evaluation



Hard evaluation requires exact coincidence; soft evaluation considers the degree of temporal association.

Hard evaluation

- Each instant is a hit or miss — no gradation.
- Detection outside the exact instant = simultaneous FP + FN.
- Simple to calculate, but too rigid for temporal events.

Soft evaluation

- Temporal proximity influences the detection score.
- Nearby detections receive partial credit proportional to distance.
- More aligned with the real operational utility of the detector.

Example: Methods A and B

Two methods detect near E1; only B detects near E2. In hard evaluation: A and B have the same recall if both hit E1 exactly — but B is clearly superior for covering E2. In soft evaluation: B receives higher TPs for covering two events, reflecting its true superiority.

SoftED: intuition

Relaxing classical metrics

SoftED adapts precision, recall and F1 to account for temporal tolerance, while maintaining the same interpretable vocabulary.

Credit proportional to proximity

Each detection receives a weight between 0 and 1 based on the distance to the nearest event within the tolerance window.

Compatibility with classical metrics

When k = 0, SoftED reduces to classical hard evaluation — it is a generalization, not a replacement.

Temporal Membership Function

Definition

The function μej(t) measures the temporal proximity between a detection at time t and the event ej.

$$\mu_{e_j}(t_{d_i}) \in [0,1], \quad \mu_{e_j}(e_j) = 1$$

Behavior

- Maximum value (1) at the exact instant of the event.
- Decreases monotonically with temporal distance.
- Falls to zero outside the tolerance window k.

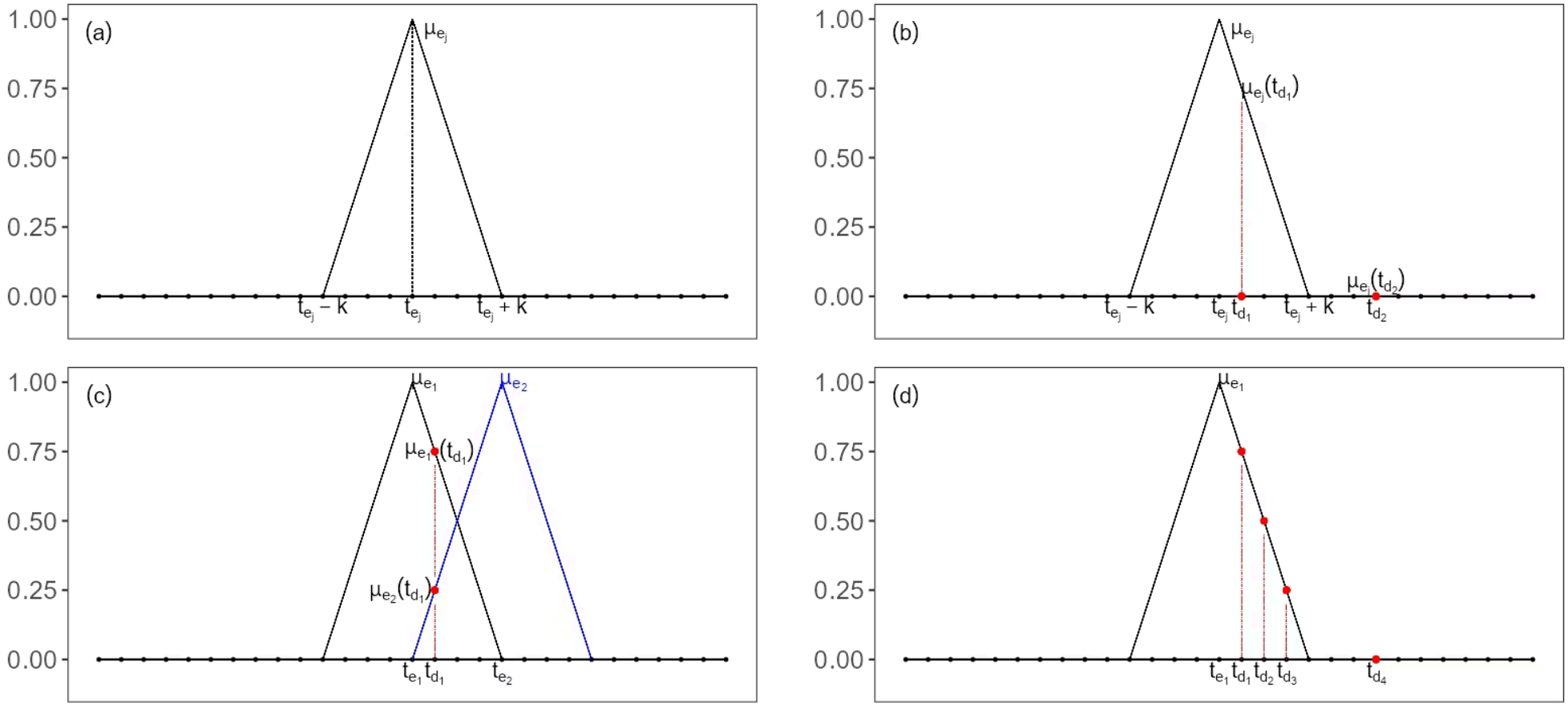
Possible Function Shapes

- **Linear:** decay proportional to distance.
- **Gaussian:** smooth decay centered on the event.
- **Rectangular:** full credit within the window, zero outside (equivalent to hard evaluation with tolerance).

Parameters

ej = event instant; k = tolerance window; tdi = detection instant i.

SoftED mapping



SoftED maps detections to events using temporal membership and association constraints.

Algorithm: SoftED mapping

Definition

Associates each detection with the association that maximizes temporal membership in groups of overlapping windows. Applies uniqueness constraints.

Membership function

$$\mu_{e_j}(t_{d_i}) = \max(0, 1 - |t_{d_i} - e_j|/k)$$

Constraints

- Each detection → at most 1 event.
- Each event → at most 1 representative detection (the one with highest μ).

Parameters

- k = tolerance window
- μ ∈ [0,1] = temporal membership

Pseudocode

```
INPUT: events E[1..m], detections D[1..p], tolerance k
PRE-CONDITION: k > 0; E and D are lists of time instants

STEP 1 — For each event ej:
    create window [ej − k, ej + k]
STEP 2 — Merge windows of events that overlap,
    forming independent temporal groups
STEP 3 — For each group:
    D_g ← detections within the merged window
    E_g ← events within the merged window
STEP 4 — Compute membership matrix:
    Mu[i,j] = max(0, min((di−(ej−k))/k, ((ej+k)−di)/k))
STEP 5 — Associate detections and events:
    if |D_g|=1 or |E_g|=1 → keep highest membership
    if |D_g|>1 and |E_g|>1 → solve optimal assignment
    maximizing sum of Mu with HungarianSolver
STEP 6 — Sum associated scores:
    TPs = sum of associated memberships
    FPs = sum(1 − score) over evaluated detections
    FNs = |E| − TPs

OUTPUT: soft TPs, FPs, FNs and temporal mapping
POST-CONDITION: associations respect conflicts within each temporal group
```

SoftED Constraints

One detection, one event

A detection should not be rewarded by multiple events at the same time — each detection is associated with the closest event within the window.

One event, one representative detection

Multiple detections should not accumulate credit for the same event — only the detection with the highest membership degree represents the event.

Consistency guarantee

These constraints ensure that soft TPs, FPs, and FNs sum correctly and that the metrics remain interpretable.

TP, FP, TN and FN in soft version

Soft TPs

Sum of temporal memberships of the representative detections for each event. TPs ∈ [0, |E|] where |E| is the number of events.

Soft FNs

Measures uncovered events: FNs = |E| − TPs. Events with no representative detection contribute 1 to FNs.

Soft FPs

Detections not associated with any event (outside every tolerance window). FPs = |D| − number of representative detections.

Soft TNs

Instants with no event and no detection. Calculated as total instants − (TPs + FPs + FNs).

The soft versions preserve the relation: TPs + FNs = |E| (total number of real events).

Soft precision, recall and F1

Soft formulas

Soft Precision

$$\text{soft_precision} = \frac{TPs}{TPs + FPs}$$

Soft Recall

$$\text{soft_recall} = \frac{TPs}{TPs + FNs} = \frac{TPs}{|E|}$$

Soft F1

$$\text{soft_F1} = \frac{2 \cdot \text{soft_precision} \cdot \text{soft_recall}}{\text{soft_precision} + \text{soft_recall}}$$

Properties

- The formulas are identical to the classical ones — only TP, FP, FN are replaced by their soft versions.
- The scale remains in [0, 1] and is interpretable.
- When $k = 0$, soft_precision = precision and soft_recall = recall (reduction to the hard case).

Advantage

Temporal proximity now influences the result without changing the evaluation vocabulary.

Algorithm: Soft Metrics Calculation

Definition

Computes soft precision, soft recall, and soft F1 from the TPs, FPs, and FNs values obtained by the SoftED mapping.

Formulas

$$soft_precision = TPs/(TPs + FPs)$$

$$soft_recall = TPs/|E|$$

$$soft_F1 = 2 \cdot sp \cdot sr/(sp + sr)$$

Properties

- TPs ∈ [0, |E|] — fractional value.
- When k = 0 → reduces to the hard case.
- sp = soft precision; sr = soft recall.

Pseudocode

```
INPUT: TPs (soft), FPs (soft), |E| = number of real events
PRE-CONDITION: TPs ≥ 0; FPs ≥ 0; |E| > 0  (obtained by the SoftED mapping algorithm)

STEP 1 — Compute soft precision
 IF TPs + FPs > 0:
  soft_precision ← TPs / (TPs + FPs)
 ELSE:
  soft_precision ← 0   [no detection emitted]
STEP 2 — Compute soft recall
 IF |E| > 0:
  soft_recall ← TPs / |E|
 ELSE:
  soft_recall ← 1   [no events, recall trivially 1]
STEP 3 — Compute soft F1
 IF soft_precision + soft_recall > 0:
  soft_F1 ← 2 · soft_precision · soft_recall
        / (soft_precision + soft_recall)
 ELSE:
  soft_F1 ← 0
STEP 4 — (Optional) Compute soft Fβ
 soft_Fβ ← (1 + β²) · soft_precision · soft_recall
       / (β² · soft_precision + soft_recall)

OUTPUT: soft_precision, soft_recall, soft_F1, soft_Fβ
POST-CONDITION: all values ∈ [0, 1]
```

SoftED vs Hard Metrics

Hard evaluation

- Requires exact match between detection and event.
- Near miss = simultaneous FP + FN.
- Result: penalizes useful detectors and underestimates real performance.

SoftED

- Rewards nearby detection with credit proportional to temporal relevance.
- Near miss = partial credit, not full penalty.
- Result: evaluation more aligned with the operational utility of the detector.

The difference between hard and soft is more pronounced when events are rare and detections are close but not exact.

Why SoftED Preserves Interpretability

Same vocabulary

The result remains in precision, recall and F1 — no need to learn new metrics or scales.

Consistent generalization

The metric is an extension of the classical ones: k = 0 reproduces exactly the hard evaluation, without breaking compatibility.

Direct comparison

It is possible to compare hard and soft evaluation in the same experiment, quantifying the impact of temporal tolerance.

Relationship with Harbinger

Availability

SoftED is implemented in the Harbinger package, available for direct use in experiments.

Integrated comparison

Allows comparing hard and soft evaluation within the same experimental workflow, without the need for manual implementation.

Theory-practice bridge

It is the direct connection between the temporal tolerance concepts presented in this tutorial and practical experimentation.

Point vs. Interval Events

Point events

Occur at a single instant in time.

Evaluation: check whether the instant was detected (with or without tolerance).

Example: voltage spike, instantaneous sensor failure.

Interval events

Have a start, end, and duration — they form a temporal segment.

Evaluation: consider coverage, detection delay, and overlap between the real and detected segment.

Example: equipment degradation period, prolonged anomaly episode.

Metrics for interval events must handle partial coverage, late onset, and exaggerated extension of detection.

Point-by-point evaluation can distort

Duration dominance

A long event has many points and can dominate the metric, artificially inflating performance scores.

Partial coverage

A small detection within an interval may be sufficient in some applications, but insufficient in others that require full coverage.

Need for segment-oriented metrics

Evaluating intervals point-by-point ignores the semantics of the event as a unit — metrics that treat the segment as a whole are needed.

Point-Adjusted F-score (PA)

Definition

If any point within the actual segment is detected, the entire segment is considered detected (TP).

Adjustment Formula

For each actual segment Sj: if $\exists$ tdi $\in$ Sj such that detector(tdi) = 1, then all points of Sj are marked as TP.

Parameters

Sj = actual segment j; tdi = detection instant i.

Advantages

- Generous for partial detections — a single detection within the segment is sufficient.
- Simple to calculate and interpret.

Limitations

- May overestimate performance when coverage matters.
- Does not penalize detections that cover only a minimal fraction of the event.
- Does not account for detection delay within the segment.

Delay-thresholded Point-Adjusted F-score (PA-k)

Definition

Extension of PA that adds a maximum delay window k.

The event is only counted as detected if there is a detection within the first k steps of the segment.

Formula

For each segment Sj starting at ts: the event is TP only if $\exists$ tdi $\in$ [ts, ts+k] such that detector(tdi) = 1.

Parameters

k = maximum delay window; ts = start of the real segment.

When to use

- Scenarios where late detection does not solve the problem (e.g., security alerts).
- When there is an operational cost associated with detection delay.

Limitations

- Still does not penalize detections that cover only the beginning of the segment.
- The choice of k is domain-sensitive.

Segment-wise F-score

Definition

Each real segment is treated as an indivisible unit.

A single detection within the segment is enough to generate a true positive.

Difference from PA

In PA, all points in the segment are marked as TP.
In segment-wise, the entire segment counts as 1 TP — there is no inflation due to duration.

Advantages

- Does not strongly penalize the exaggerated extension of a detected interval.
- Treats events of different durations in an equitable manner.

Limitations

- Does not evaluate partial coverage — a minimal detection is equivalent to full coverage.
- Does not consider delay within the segment.

Range-based F-score

Definition

Evaluates partial coverage of real events and detections using interval overlap.

Components

$$\text{recall_local}(S_j, D_j) = \frac{|S_j \cap D_j|}{|S_j|}$$

$$\text{precision_local}(S_j, D_j) = \frac{|S_j \cap D_j|}{|D_j|}$$

Parameters

Sj = real segment *j*; *Dj* = associated detected segment; |·| = interval duration.

Advantages

- Captures partial coverage — a detection covering 70% of the event receives proportional credit.
- Penalizes excessively long detections via local precision.
- More informative than PA and segment-wise for interval events.

Limitations

- More complex to compute and interpret.
- Requires association between real and detected segments.

Algorithm: Range-based Evaluation of Interval Events

Definition

Evaluates partial coverage between real and detected segments using interval overlap. Computes local recall and precision per segment pair.

Formulas

$$\text{recall_local}(S_j, D_j) = \frac{|S_j \cap D_j|}{|S_j|}$$

$$\text{precision_local}(S_j, D_j) = \frac{|S_j \cap D_j|}{|D_j|}$$

$$F1_range = \text{harmonic mean of recall_local and precision_local}$$

Parameters

- *Sj* = real segment *j* = [s_start, s_end]
- *Dj* = detected segment associated with *Sj*
- |·| = interval duration (number of time steps)

Pseudocode

```
INPUT: real segments S[1..m], detected segments D[1..p]
PRE-CONDITION: each segment is an interval [start, end]; start ≤ end

STEP 1 — Associate each real segment with the detected segment having greatest overlap
  For each Sj in S:
    For each Di in D:
      overlap[j][i] ← |Sj ∩ Di|   [= max(0, min(Sj.end, Di.end) − max(Sj.start, Di.start) + 1)]
    i* ← argmax_i overlap[j][i]
    IF overlap[j][i*] > 0 → associate Sj with Di*
    ELSE → Sj not detected (recall_local = 0)
STEP 2 — Compute local metrics for each pair (Sj, Di*)
  recall_local[j]    ← |Sj ∩ Di*| / |Sj|
  precision_local[j] ← |Sj ∩ Di*| / |Di*|
STEP 3 — Aggregate global metrics
  recall_global    ← mean(recall_local[j]) for all j
  precision_global ← mean(precision_local[j]) for all j
STEP 4 — Compute range-based F1
  F1_range ← 2 · precision_global · recall_global
             / (precision_global + recall_global)

OUTPUT: recall_global, precision_global, F1_range
POST-CONDITION: all values ∈ [0, 1]
```

Online Evaluation Is Different

Partial View of the Series

The detector does not see the full series — it processes sequential batches and makes decisions with limited information.

Dynamic Detection

The decision about a given time point can change as new batches arrive — the detector may revise its assessment.

Time as a Performance Dimension

The moment of detection is an essential part of performance — detecting early has a different value than detecting late.

Offline metrics do not capture these dimensions — online evaluation requires specific metrics such as detection probability and detection lag.

Types of online detection

Early

Detection occurs before the event — may indicate a precursor signal. Useful in preventive applications, but can be a FP if the event does not occur.

Late

Detection occurs after the event — in online detection, it may be operationally useless if the action window has already passed.

Inconsistent

The method changes its decision as the window evolves — detects in one batch and revokes in another. Indicates detector instability.

Example: Methods A and B in online evaluation

As new observations arrive in batches, Method A detects the event with a delay of 3 batches (**late**), while Method B detects at the first opportunity (lag=0) but revokes the decision in the next batch (**inconsistent**). The experiment image shows how detection probability and lag diverge between the two methods.

Detection Probability

Definition

Measures the probability of an observation being detected across the available opportunities — considers all batches in which the observation was available.

Formula

$$P_{det}(o_i) = \frac{\text{number of batches in which } o_i \text{ was detected}}{\text{number of batches in which } o_i \text{ was available}}$$

Interpretation

P_det = 1 means the observation was detected in every available batch.
P_det = 0 means it was never detected.

Detection probability captures the consistency of detection over time, not just whether detection occurred at all.

Detection Lag

Definition

Measures the delay between the first availability of an observation and its first detection.

Formula

$$\text{lag}(o_i) = \text{first batch in which } o_i \text{ was detected} - \text{first batch in which } o_i \text{ was available}$$

Units

Can be expressed as a number of batches or as a number of observations.

Interpretation

- **lag = 0** → immediate detection at the first opportunity.
- **lag > 0** → delayed detection — the larger the value, the later the method reacted.
- **lag = ∞** (or undefined) → observation was never detected.

Importance

The smaller the lag, the earlier the method reacted to the event — essential in real-time applications.

Algorithm: Detection Lag Calculation in Online Evaluation

Definition

Measures the delay between the first availability of an observation and its first detection across sequential batches.

Formula

$$lag(oi) = b_det(oi) - b_avail(oi)$$

Parameters

- oi = observation of interest (event)
- b_avail(oi) = first batch in which oi was available
- b_det(oi) = first batch in which oi was detected
- lag = ∞ if oi was never detected

Units

Expressed as number of batches or number of observations.

Pseudocode

```
INPUT: observation of interest oi, sequence of batches B[1..T],
       detector decisions det[b][t] for each batch b and time step t
PRE-CONDITION: oi is a real event; B is chronologically ordered

STEP 1 — Identify first availability batch
  b_avail ← first b such that oi ∈ B[b]
  IF b_avail does not exist → ERROR: oi never available
STEP 2 — Identify first detection batch
  b_det ← ∞   [initialize as not detected]
  For b from b_avail to T:
    IF det[b][oi] = 1:
      b_det ← b
      STOP
STEP 3 — Compute lag
  IF b_det < ∞:
    lag(oi) ← b_det − b_avail
  ELSE:
    lag(oi) ← ∞   [observation never detected]
STEP 4 — (Optional) Aggregate average lag over all events
  lag_mean ← mean(lag(oi)) for oi with lag < ∞

OUTPUT: lag(oi) for each observation, lag_mean
POST-CONDITION: lag(oi) ≥ 0 for all detected oi
```

Batch size, memory and evaluation window

Batch size

Alters the granularity of evaluation — smaller batches allow faster detection, but with less context available.

Partial memory

Limits how much of the historical series the detector can use — affects the ability to detect long-duration patterns.

Evaluation window

Defines the temporal horizon considered to calculate detection probability and lag — larger windows capture more detection opportunities.

The combination of batch size, memory, and evaluation window defines the online experimental protocol and directly affects the metrics.

Benchmark Quality

Influence on Conclusions

The choice of benchmark strongly influences the conclusion about a method — poor benchmarks can reverse the performance ranking.

Poor Benchmarks Favor Simple Methods

Trivial anomalies, unrealistic event density, or poorly labeled ground truth can make simple methods appear superior.

Quality Criteria

A good benchmark should have: non-trivial events, realistic density, accurate labels, and diversity of temporal patterns.

Common problems in benchmarks

Trivial anomalies

Favor overly simple methods — any basic detector gets it right, without discriminating real quality.

Unrealistic event density

Artificially skews precision and recall — too many events inflate recall; too few inflate precision.

Poorly labeled ground truth

Compromises any metric — errors in labels contaminate the entire evaluation.

Temporal concentration

Many anomalies at the end of the series can bias evaluation — the detector may learn the label distribution pattern.

Recent Benchmarks

UCR Anomaly Archive

A curated collection of time series for anomaly detection evaluation. Focused on label quality and diversity of patterns.

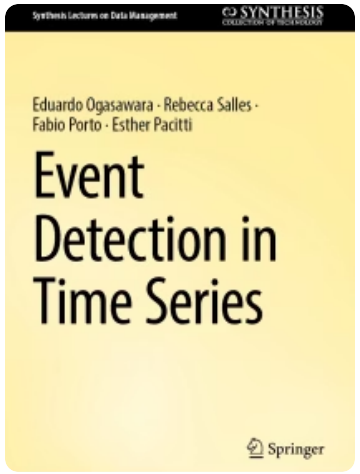
TimeEval

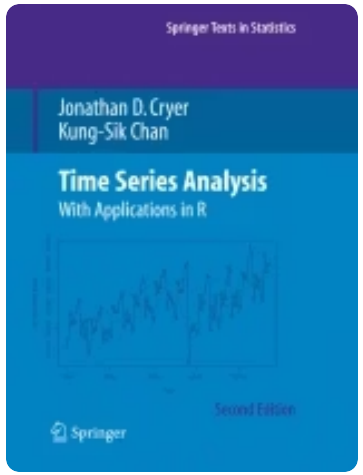
Toolkit for systematic comparison of anomaly detection algorithms in time series. Supports multiple metrics and evaluation protocols.

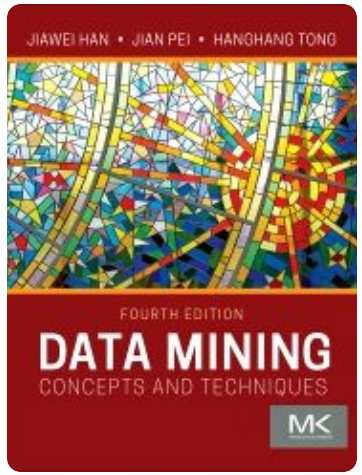
Recent Advances

Modern benchmarks address label quality, interval-based events, time-tolerant metrics, and online evaluation protocols.

Bibliographic References







Event Detection in Time Series

Ogasawara, E.; Salles, R.; Porto, F.; Pacitti, E. Springer Nature Switzerland, 2025.

Time Series Analysis: With Applications in R

Cryer, J. D.; Chan, K.-S. Springer, 2008.

Data Mining: Concepts and Techniques

Han, J.; Pei, J.; Tong, H. Morgan Kaufmann, 4th ed., 2022.